Triplet-Watershed for Hyperspectral Image Classification

Aditya Challa, Sravan Danda, *Member, IEEE*, B. S. Daya Sagar, *Senior Member, IEEE*, and Laurent Najman, *Senior Member, IEEE*

***Abstract*— Hyperspectral images (HSIs) consist of rich spatial and spectral information, which can potentially be used for several applications. However, noise, band correlations, and high dimensionality restrict the applicability of such data. This is recently addressed using creative deep learning network architectures, such as ResNet, spectral-spatial residual network (SSRN), and attention-based adaptive spectral-spatial kernel residual networks (A2S2K). However, the last layer, i.e., the classification layer, remains unchanged and is taken to be the softmax classifier. In this article, we propose to use a watershed classifier. Watershed classifier extends the watershed operator from Mathematical Morphology for classification. In its vanilla form, the watershed classifier does not have any trainable parameters. In this article, we propose a novel approach to train deep learning networks to obtain representations suitable for the watershed classifier. The watershed classifier exploits the connectivity patterns, a characteristic of HSI datasets, for better inference. We show that exploiting such characteristics allows the Triplet-Watershed to achieve state-of-art results in supervised and semi-supervised contexts. These results are validated on Indian Pines (IP), University of Pavia (UP), Kennedy Space Center (KSC), and University of Houston (UH) datasets, relying on simple convnet architecture using a quarter of parameters compared to previous state-of-the-art networks. The source code for reproducing the experiments and supplementary material (high-resolution images) is available at https://github.com/ac20/TripletWatershed_Code.**

***Index Terms*— Classification, deep learning, hyperspectral imaging, triplet loss, watershed.**

Manuscript received May 14, 2021; revised August 29, 2021; accepted September 3, 2021. Date of publication October 6, 2021; date of current version January 26, 2022. The work of Aditya Challa was supported in part by the Indian Institute of Science for the Raman Fellowship and in part by Birla Institute of Technology and Science (BITS)-Pilani, K. K. Birla Goa Campus, under Grant BPGC/RIG/2021-22/09-2021/02. The work of Sravan Danda was supported by the Research Initiation Grant provided by BITS-Pilani, K. K. Birla Goa Campus, under Grant BPGC/RIG/2020-21/11-2020/01. The work of B. S. Daya Sagar was supported in part by the DST-ITPAR-Phase-IV Project sanctioned under Grant INT/Italy/ITPARIV/Telecommunication/2018. The work of Laurent Najman was supported by the Programme d'Investissements d'Avenir under Grant LabEx BEZOUT ANR-10-LABX-58. *(Corresponding author: Aditya Challa.)*

Aditya Challa and Sravan Danda are with Anuradha and Prashanth Palakurthi Centre for Artificial Intelligence Research (APPCAIR), Department of Computer Science and Information Systems, Birla Institute of Technology and Science (BITS) Pilani, K. K. Birla Goa Campus, Sancoale, Goa 403726, India (e-mail: aditya.challa.20@gmail.com; sravan8809@gmail.com).

B. S. Daya Sagar is with the Systems Science and Informatics Unit, Indian Statistical Institute, Bengaluru, Karnataka 560059, India (e-mail: bsdsagar@yahoo.co.uk).

Laurent Najman is with LIGM, Univ. Gustave Eiffel, CNRS, F-77454 Marne-la-Vallée, France (e-mail: laurent.najman@esiee.fr).

This article has supplementary downloadable material available at https://doi.org/10.1109/TGRS.2021.3113721, provided by the authors.

Digital Object Identifier 10.1109/TGRS.2021.3113721

I. Introduction

HYPERSPECTRAL imaging has several applications ranging across different domains [1]. It has seen applications in earth observations [2], and land cover classification [3], etc. Hyperspectral datasets have rich information both spatially and spectrally. However, spectral and spatial correlations make a lot of such information redundant. One can obtain efficient representations using techniques such as band selection [4], [5], subspace learning [6], [7], multimodal learning [8], and low-rank representation [9].

Large number of bands, spatial and spectral feature correlations, and curse of dimensionality make Hyperspectral image (HSI) classification challenging. Conventional approaches use hand-crafted features with techniques such as scale-invariant feature transform (SIFT) [10], sparse representation [11], principal component analysis (PCA) [12], and independent component analysis [13]. Classic approaches to classification such as support vector machines (SVM) [2], neural networks [14], and logistic regression [15] aimed at exploiting the spectral signatures alone. Using spatial features have been extremely useful to obtain better representations and higher classification accuracies [16]–[18], which the classic approaches ignore. Multiple kernel learning [19]–[21] use hand-designed kernels to exploit the spectral–spatial interactions. Deep learning approaches, especially CNNs, have been adapted to exploit the spectral–spatial information. Chen *et al.* [22] proposed a 3D-CNN feature extractor to obtain combined spectral–spatial features. Yang *et al.* [23] adapted CNN to a two-branch architecture to extract joint spectral-spatial features. Ben Hamida *et al.* [24] used 3-D volumes to extract spectral-spatial features, which may be improved using multiscale approaches [25]. Spectral–spatial residual network (SSRN) proposed in [26] uses residual networks to remove the declining accuracy phenomenon. Residual spectral–spatial attention networks (RSSANs) [27] exploit the concept of attention to improve on SSRNs. Roy *et al.* [28] proposed attention-based adaptive spectral-spatial kernel residual networks (A2S2K) by exploiting adaptive kernels. Hong *et al.* [29] used graph convolution networks (GCN) and Paoletti *et al.* [30] used capsule networks. Most of these approaches tackle the problem of HSI classification by considering novel architectures. Another prominent direction of research focusses on using unlabelled data for improving classification accuracies, referred to as semi-supervised learning. Hong *et al.* [31] and [32] used

1558-0644 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 04:30:08 UTC from IEEE Xplore. Restrictions apply.

hyperspectral data for improving inference on multispectral data. Hong *et al.* [29] proposed a semi-supervised approach to exploit multimodal data for better inference. GCNs have also been used to obtain state-of-art results on hyperspectral classification as evidenced by S2GCN [33] and dual clustering GCN (DC-GCN) [34]. Other approaches include local constraint-based sparse manifold hypergraph learning (LC-SMHL) [35], self-adaptive manifold discriminant analysis (SAMDA) [36], DLPNet [37], and adaptive residual convolutional neural network (ARCNN) [38].

In this article, we take a different route to propose a novel classifier based on the watershed operator. Watershed operator from Mathematical Morphology [39], [40] has been widely used for image segmentation, and, in particular, for HSIs [41], [42]. Tarabalka *et al.* [42] combined (by majority voting) several watersheds computed on gradients of different bands. They observe that class-specific accuracies were improved by using the spatial information in the classification for almost all the classes, a result that we are going to confirm in the present article. To our knowledge, watersheds have not been used in conjunction with current state-of-art neural networks in the context of HSIs. We propose a novel approach to achieve this in the current article.

In [43], the watershed operator is adapted to edge-weighted graphs. It is shown there that the watershed is closely related to the minimum spanning tree (MST) of the graph. Watersheds have also been highly successful as a post-processing tool for image segmentation [44]–[46]. Funke *et al.* [47] learnt a representation suitable for MST-based classification. Wolf *et al.* [48] learnt a representation suitable to mutex-watershed, a different version of the watershed.

Departing from images, in our previous work [49] we have proposed to use the watershed operator as a generic classifier. We showed that it obtains a maximum margin partition similar to the support vector machine. We further showed that ensemble watersheds obtain comparable performance to other classifiers such as random forests. In this article, we propose a novel approach, simple and efficient, called Triplet-Watershed to learn representations (also known as embeddings) suitable for the watershed classifier.

Why Watershed Classifier? Previous work on HSI classification, as discussed above, establish that one must use both spatial and spectral aspects to obtain good classifiers. They achieve this with creative approaches to design neural networks such as adaptive kernels and attention mechanism. However, most of these still use conventional softmax classifier. The watershed classifier naturally uses spatial information for inference. Thus, it allows us to use simpler networks for representation. Table I shows the overall accuracy (OA) scores obtained by our approach and other state of art methods. It also shows the number of parameters used. Observe that Triplet-Watershed parameters are just 25% of those of the current state-of-art (A2S2K) approach.

The main contributions of this article are the following.

1) We propose a novel approach, namely the Triplet-Watershed, to learn a representation suitable to the watershed classifier. This representation is referred to as watershed representations in the rest of this article.

TABLE I

OA VERSUS NUMBER OF PARAMETERS. OBSERVE THAT THE PROPOSED METHOD HAS VERY LESS NUMBER OF PARAMETERS BUT OUTPERFORMS THE CURRENT STATE-OF-THE-ART APPROACHES. IP INDICATES INDIAN PINES DATASET. UP DENOTES UNIVERSITY OF PAVIA DATASET AND KSC INDICATES THE KENNEDY SPACE CENTRE DATASET

	# params	IP	UP	KSC
A2S2K [28]	370.7K	98.66	99.85	99.34
SSRN [26]	364.1K	98.38	99.77	99.29
ENL-FCN [50]	113.9K	96.15	99.76	99.25
ResNet34 [51]	21.9M	92.44	97.38	79.73
Triplet-Watershed	**87.6K**	**99.57**	**99.98**	**99.72**

2) The Triplet-Watershed achieves state-of-art results on the hyperspectral datasets with very simple networks, using much fewer parameters than the previous state-of-the-art approaches as described in Table I.
3) The same Triplet-Watershed approach can be used for both supervised and semi-supervised tasks without any modification, still leading to state-of-the-art results compared to previous approaches.
4) The framework used here to obtain representations is not restricted to watershed classifiers. It can be extended to use with other classifiers such as random forest or k-nearest neighbors as well, although watershed results outperform other classifiers on our datasets.
5) The main insight of our article is that enforcing spatial connectivity (achieved thanks to the watershed classifier) during the training is a relevant constraint for hyperspectral classification.

Overview: Section II reviews the watershed classifier and the required terminology for the rest of this article. In Section III, we design the neural net (NN) and the training procedure to learn watershed representations. Section IV provides empirical analysis.

II. WATERSHED CLASSIFIER

The watershed classifier is defined on an edge-weighted graph. We follow the exposition as given in [49]. $G = (V, E, W)$ denotes the edge-weighted graph. Here, V denotes the set of vertices, E denotes the set of edges which is a subset of $V \times V$ and $W : E \to \mathbb{R}^+$ denotes the edge weight assigned to each edge. We assume that the edge weights are all positive in this article.

The (two-class) classification problem on the edge-weighted graph is stated as—let $X_0, X_1 \subset V$ denote the labeled subset of vertices labeled 0 and 1, respectively. Classification problem requires a partition of $V = M_0 \cup M_1$ with $M_0 \cap M_1 = \emptyset$. With an additional constraint that $X_0 \subset M_0$ and $X_1 \subset M_1$. Here M_0 denotes all the vertices labeled 0 after classification and M_1 denotes all the vertices labeled 1. We also assume there exists a dissimilarity measure $\rho(x, y)$ between two vertices $x, y \in V$. This measure extends to subsets as

$$\rho(X, Y) = \min_{x \in X, y \in Y} \rho(x, y) \tag{1}$$

where X, Y are arbitrary subsets of V. Observe that there exist several partitions of $V = M_0 \cup M_1$ which satisfy the above

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 04:30:08 UTC from IEEE Xplore. Restrictions apply.

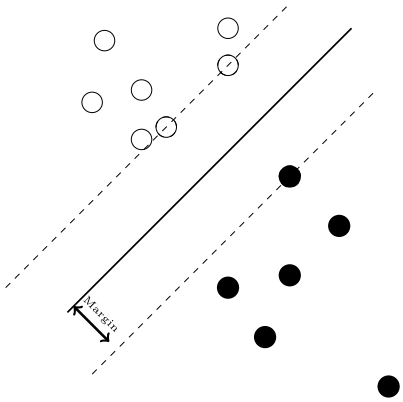


Fig. 1. Illustration of maximum margin for SVM [49]. The key observation is—the margin is defined as the minimum distance between the training point labeled 0 and what would be labeled 1 after classification, and vice versa. The aim of the (linear) SVM classifier is to obtain a decision boundary that maximizes the margin. This can be extended to obtain a maximum-margin partition on an edge-weighted graph. Using (2), a solution of this is provided by the watershed classifier.

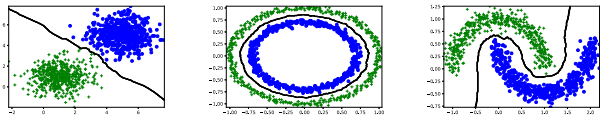


Fig. 2. Watershed boundaries [49]. Observe that in all these cases the boundary is in-between the classes. Also, it is in the middle of the zero density (no points exist) regions. This maximizes the margin between the boundaries and the classes. This is consistent with the maximum margin principle of SVM. (a) Two Gaussian blobs dataset. (b) Two circles dataset. (c) Two moons dataset.

conditions. Of these partitions, we only use the Maximum Margin Partitions, i.e., the partitions which maximize

$$\min\{\rho(X_0, M_1), \rho(X_1, M_0)\}. \tag{2}$$

This follows from the maximum margin principle of SVM. From Fig. 1, a key observation can be made—the margin for the SVM is the minimum distance between training point labeled 0 and what would be labeled 1 after classification, and vice versa. Linear SVM tries to obtain the boundary to maximize this margin. This can be extended to the edge-weighted graphs using (2).

The Watershed Classifier is defined by considering the dissimilarity measure to be

$$\rho(x, y) := \rho_{\max}(x, y) = \min_{\pi \in \Pi(x,y)} \max_{e \in \pi} W(e) \tag{3}$$

where π denotes a specific path between x, y. Π denotes the set of all possible paths. $\rho_{\max}$ is sometimes referred to as pass value.

If each edge-weight indicates the height of the corresponding edge, then $\rho_{\max}(x, y)$ indicates the minimum height one has to climb to reach y from x. When the points belong to a Euclidean space, the edge weight is given by Euclidean distance. That is, the edge weight indicates the distance between the points. Hence, $\rho_{\max}(x, y)$ would indicate the minimum "jump" one has to make to reach y from x. Thus, the boundaries (in cases where the classes are separable) would be along the low-density regions between classes. This is illustrated in Fig. 2. In all the cases, the boundary is between the classes such that we have the maximum margin. This is consistent with the maximum margin principle of SVM.

Remark: One can replace the pass value in (3) with several other measures, leading to different classifiers. Detailed analysis of replacing pass value with other measures is out of scope for the present article and may be considered for future work. For instance, using the image foresting transform (IFT) [52] leads to a classifier similar to the one proposed in [53]. Few such techniques are discussed in [49].

Given the edge-weighted graph, the Watershed algorithm extends the Maximum Margin Partition principle to several classes and obtains the labels using the UnionFind data structure. This is described in Algorithm 1.

Algorithm 1 Watershed Clustering Algorithm [49]

Input: edge-weighted graph $G = (V, E, W)$. A subset of labeled points $V_l \subset V$.
Output: Labels for each of the vertices L

```
1: Sort the edges E in increasing order w.r.t W.
2: Initialize the union-find data structure UF,
3: for e = (e_x, e_y) in sorted edge set E do
4:   if both e_x and e_y are labeled then
5:     do nothing
6:   else
7:     UF.union(e_x, e_y)
8:     Assign same label for e_x and e_y.
9:   end if
10: end for
11: Label each vertex of the connected component using labels V_l.
12: return Labels of the vertices.
```

Observe that step (10) is possible since each connected component would have exactly one unique label. One can see that watershed clustering is a semi-supervised algorithm, in the sense that it propagates the known labels to points with unknown labels.

To illustrate the watershed classifier consider the simple edge-weighted graph in Fig. 3(a). The two distinct colors indicate two classes. No color indicates that the vertex is not yet labeled. In the first step, the least edge-weight is 1. Adding all these edges [thick edges in Fig. 3(b)] gives four distinct components. Each component is labeled according to the label present within the component. In case there exists no label, then label assignment is not yet carried out. We then add the edges with weight 2, and label the points accordingly. Observe that there are no more unlabelled points and hence the algorithm terminates.

In practice, it has been observed that ensemble techniques improve the robustness of watershed classifier. This is achieved using only a subset of labeled points and only a subset of features and taking the weighted average. Details can be found in [49]. We refer to these two approaches as single watershed classifier and ensemble-watershed classifier.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 04:30:08 UTC from IEEE Xplore. Restrictions apply.

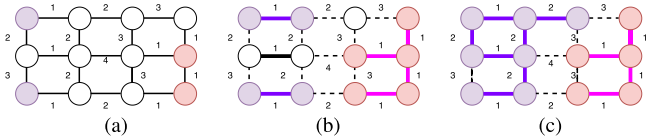


Fig. 3. Illustrating the watershed classifier. (a) Edge-weighted. The two distinct colors indicate two different classes. No color indicates that the vertex is not yet labeled. (b) Graph obtained by adding edges with weight 1. Each vertex is given a label accordingly. (c) Graph obtained by adding the edges with weight 2 and Propagating the labels. Observe that all the points are now labeled and hence the algorithm terminates. (a) Original graph. (b) Intermediate step. (c) Watershed labels.

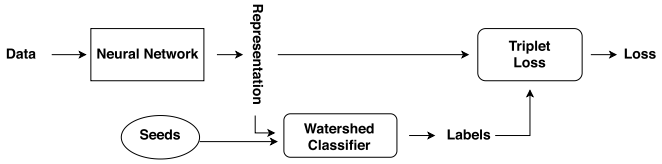


Fig. 4. Schematic of learning representations for the watershed classifier. Using a generic neural network we obtain the representation for the dataset. These representations are fed into the watershed classifier to obtain the labels using the seeds. Using the labels and the representation, we use triplet loss to compute the loss and also for obtaining the parameters for the neural network. Observe that the watershed classifier needs to be computed at every epoch.

III. Learning Representations for the Watershed Classifier

Section II describes how one can obtain the labels using the watershed classifier. In [49], it was shown that this compares reasonably well to other classifiers such as SVM, random forests, etc. However, observe that this classifier has no trainable parameters. In this section, we develop an approach to train a neural network for learning representations suitable to the watershed classifier.

A key observation is—Watershed classifier reduces the distances within each component and increases the distance across components. This leads to the schematic in Fig. 4. First, we use a generic neural network to obtain the representations for the dataset. These representations, along with a subset of labeled points, are used with the watershed classifier to obtain the labels. Using these labels, we obtain a metric-learning loss to decide if two pixels are either in the same component (same label) of the watershed or in two different components (different label). More precisely, we use triplet loss [54], [55] to learn the watershed representation. For training, this cost is minimized using standard autograd packages such as PyTorch.

Why Schematic in Fig. 4 Learns Watershed Representations? Triplet loss function uses {(anchor, positive, negative)} triplets for computation of the cost. It compares an anchor-input to a positive-input and a negative-input. The distance from the anchor-input to the positive-input is minimized, and the distance from the anchor-input to the negative-input is maximized using the cost

$$\min\{d(\text{anchor}, \text{positive}) - d(\text{anchor}, \text{negative}) + \alpha\}_+ \quad (4)$$

where $\{*\}_+$ denotes the function $\max\{0, *\}$. By enforcing the order of distances, triplet loss models embed in the way that a pair of samples with the same label are smaller in distance than those with different labels. When watershed labels are used to obtain {(anchor, positive, negative)} triplets, this leads to representations that are compatible with the watershed classifier.

Remark (Supervised Versus Semi-Supervised): Recall that the watershed classifier uses a subset of training points (referred to as seeds) to obtain the labels of other training points. These labels are then used to train the network with triplet loss. However, in the case of semi-supervised learning, unlabelled data is also available at train time. These points can be labeled and be used to train the network. In this article, we use the semi-supervised approach, randomly choosing some seeds for the watershed classifier that iteratively propagates their labels to their most resembling neighbors, obtaining the connected components. Hence, the combination of watershed clustering and triplet loss ensures that points with the most resembling representations are indeed clustered together, in the same connected component.

A. Training Dynamics

To summarize the entire training procedure of Triplet-Watershed, at each epoch.

1) Obtain the representations for all the points using the neural network.
2) We consider a randomly chosen subset of labeled points as seeds.
3) Propagate the labels to all points using the watershed classifier.
4) Use the watershed labels to generate {$(anchor, positive, negative)$} triplets.
5) Use the triplet loss to train the neural network.

Few obvious questions follow: 1) When would the training converge? and 2) What is the steady-state obtained?

Note that the training would converge when there would be no further improvement in the triplet-loss. At this stage, the out-of-box score[1] of the watershed classifier would not improve as well. This implies that—all pairs of points with the same labels and within the same component have a similar representation. Hence, we obtain 100% out-of-box accuracy[2] with watershed classifier.

Remark (Overfitting): Traditional machine learning advises against reaching 100% training accuracy as the models might be overfitting. However, recent deep learning trends point to the contrary. Several deep learning models can indeed fit random data with 100% accuracy [56]. It is still an open question to understand the generalization ability of these models. However, few observations point to the inductive bias [57] as the reason behind good generalization. In our case, the inductive bias is dictated by the graph constructed from the data.

Also, note that during training we use a single watershed classifier. While, at inference, we use an ensemble-watershed classifier. This ensures robustness during inference.

Remark (Complexity): Two main steps can be identified in the above procedure: 1) obtaining a representation of the

[1]Accuracy on the training data excluding the seeds.
[2]Here, we assume that there exists at least one seed per component.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 04:30:08 UTC from IEEE Xplore. Restrictions apply.

points and 2) propagating the labels using watershed. The time complexity for obtaining the representation is dictated by matrix multiplications with the neural network. This can easily be parallelized using GPU. An empirical study of the time taken for this is discussed in Section IV. Table II shows the actual time taken for both training and evaluation. Propagation of labels is done using binary partition trees and can be performed in quasi-linear time [58]. We use the routines available at [59] for implementation.

IV. Empirical Analysis

In this section, we explore the application of the watershed classifier to the HSI classification task. We use the standard evaluation metrics for comparison.

1) *OA:* It measures the OA across all samples, not considering the class imbalance.
2) *Average Accuracy (AA):* It measures the AA across the classes and
3) *Kappa Coefficient (κ):* It measures how well the estimates and groundtruth labels correspond, taking into account agreement by random chance.

Four datasets are used for comparison.

1) *Indian Pines (IP):* Gathered by the Airborne Visible/Infrared Imaging Spectrometer (AVIRIS) [60] sensor over the test site in North-western Indiana: This dataset contains 224 spectral bands within a wavelength range of 0.4 to 2.5×10^{-6} m. The 24 bands covering a region of water absorption are removed. The image spatial dimension is 145×145, and there are 16 classes not all mutually exclusive.
2) *Kennedy Space Center (KSC):* The KSC dataset was gathered on March 23, 1996 by AVIRIS [60] with wavelengths ranging from 0.4 to 2.5×10^{-6} m. 176 spectral bands are used for analysis after removal of some low signal-to-noise ratio SNR) bands and water absorption bands. Thirteen classes representing the various land cover types that occur in this environment are defined for the site.
3) *University of Pavia (UP):* Acquired by the ROSIS [61] sensor during a flight campaign over Pavia, northern Italy. The number of spectral bands is 103 for Pavia University and is of size 610×610 pixels. The ground truth identifies nine classes.
4) *University of Houston (UH):* This dataset was acquired over the UH campus and the neighboring urban area. This dataset was captured with a spatial resolution of 2.5 m and with 144 spectral bands in the 380–1050-nm region. This has 15 groundtruth classes. The dataset can be obtained from https://hyperspectral.ee.uh.edu/?page_id = 459.[3]

We preprocess the datasets using PCA [62] to obtain orthogonal components. We use 200 principal components for IP, 176 for KSC, 103 for UP and 144 for the UH datasets. The train/test split is obtained randomly using 10% for training and 90% for testing.

[3]Accessed on 30 April 2021.

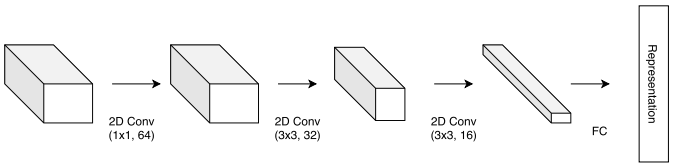


Fig. 5. Neural network architecture used for obtaining the representations. The architecture is composed of three convolution layers followed by a fully connected layer to get the representation. Batch normalization is performed before each layer for efficient training. The number of parameters is 87k.

TABLE II
Run-Times (in Seconds) of Triplet-Watershed and Other Approaches. Observe That the Running Time of Triplet-Watershed Is Comparable to Other Approaches

	Time(s)	Triplet-Watershed	A2S2K [28]	SSRN [26]
IN	Train	520.56	829.23	779.33
	Test	3.77	10.55	11.44
UP	Train	791.22	2582.31	1964.66
	Test	46.23	47.33	33.02
KSC	Train	978.25	757.46	535.20
	Test	1.58	8.37	5.84
UH	Train	1460.15	947.73	1145.38
	Test	8.74	11.55	7.85

Graph Construction: Note that the watershed classifier is defined on edge-weighted graphs. This is constructed as follows.

1) The set of vertices V is taken to be the set of all the pixels in the dataset ignoring the {labels $= 0$} class, since these points do not have any groundtruth labels.
2) The edge set E is taken to be the union of four-adjacency edges induced by the vertex set V (on the image) and edges obtained by Euclidean minimum spanning tree (EMST) [63]) for IP, UP and KSC, and K-Neighbor edges with $k = 50$ for UH dataset. The EMST and K-Neighbor edges are obtained by considering the top 32 principal components.
3) Given a representation obtained thanks to the neural network, the edge weights are computed using Euclidean distance. This representation (and hence the edge weights themselves) is updated at every epoch during training, while the edge set itself is never updated.

An illustration of the graph construction procedure is provided in Appendix A.

In all the experiments, we use the NN architecture as shown in Fig. 5. We consider a patch ($11 \times 11 \times$ #Bands) around each pixel of the input HSI, suitably padded with 0 s. We use three conv2d layers and a fully connected layer to obtain the representation. These representations are then used for watershed classification and training. All models are trained using stochastic gradient descent (SGD) with cyclic learning rates [65]. We use 40% of the training data as seeds for the watershed classifier. The default weight initialization by PyTorch [66] is used. We use 64 as the dimension for the representations. All accuracies are reported in the format mean $\times$ 100% $\pm$ stdev to be consistent with [28]. The code is available at https://github.com/ac20/TripletWatershed_Code.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 04:30:08 UTC from IEEE Xplore. Restrictions apply.

TABLE III

OA, AA, AND KAPPA (κ) VALUES ON IP DATASET USING 10% OF SAMPLES FOR TRAINING

			Classic approaches			Deep-Learning approaches		
Class	Train	Test	RF [64]	SVM [2]	Ensemble-Watershed [49]	SSRN [26]	A2S2K [28]	Triplet-Watershed
1	4	42	28.46 ± 0.061	51.22 ± 0.190	41.43 ± 0.2079	57.78 ± 0.423	97.56 ± 0.034	**100.00 ± 0.0000**
2	142	1286	56.63 ± 0.024	81.22 ± 0.037	81.07 ± 0.0202	98.37 ± 0.012	**98.62 ± 0.010**	**98.62 ± 0.0151**
3	83	747	48.42 ± 0.013	65.82 ± 0.013	71.49 ± 0.0250	97.47 ± 0.010	98.58 ± 0.006	**100.00 ± 0.0000**
4	23	214	33.49 ± 0.025	57.75 ± 0.041	45.70 ± 0.0327	99.12 ± 0.010	98.29 ± 0.014	**100.00 ± 0.0000**
5	48	435	85.21 ± 0.025	90.04 ± 0.014	92.78 ± 0.0286	97.79 ± 0.013	**99.02 ± 0.003**	97.98 ± 0.0254
6	73	657	92.64 ± 0.027	96.25 ± 0.006	98.57 ± 0.0033	98.50 ± 0.010	98.71 ± 0.010	**99.97 ± 0.0006**
7	2	26	2.67 ± 0.038	73.33 ± 0.019	99.17 ± 0.0167	66.67 ± 0.471	93.10 ± 0.097	**100.00 ± 0.0000**
8	47	431	97.67 ± 0.015	97.98 ± 0.006	98.14 ± 0.0075	96.45 ± 0.029	98.83 ± 0.016	**100.00 ± 0.0000**
9	2	18	9.26 ± 0.094	50.00 ± 0.045	37.50 ± 0.1854	56.25 ± 0.418	74.26 ± 0.038	**100.00 ± 0.0000**
10	97	875	60.91 ± 0.047	73.87 ± 0.018	85.81 ± 0.0227	98.33 ± 0.009	98.21 ± 0.016	**99.75 ± 0.0040**
11	245	2210	87.88 ± 0.019	82.90 ± 0.012	86.68 ± 0.0105	99.08 ± 0.005	99.09 ± 0.001	**99.61 ± 0.0054**
12	59	534	41.26 ± 0.030	74.91 ± 0.043	69.51 ± 0.0182	98.46 ± 0.009	98.37 ± 0.013	**99.89 ± 0.0022**
13	20	185	90.09 ± 0.040	96.94 ± 0.021	99.35 ± 0.0079	100.0 ± 0.000	99.80 ± 0.002	**100.00 ± 0.0000**
14	126	1139	95.46 ± 0.014	93.82 ± 0.010	92.59 ± 0.0085	98.63 ± 0.010	99.22 ± 0.007	**100.00 ± 0.0000**
15	38	348	41.11 ± 0.029	60.42 ± 0.044	54.48 ± 0.0396	99.24 ± 0.005	97.86 ± 0.013	**100.00 ± 0.0000**
16	9	84	79.37 ± 0.030	91.27 ± 0.054	79.29 ± 0.1163	95.63 ± 0.062	95.93 ± 0.057	**98.10 ± 0.0267**
OA	1018	9231	72.98 ± 0.006	82.00 ± 0.006	83.75 ± 0.0076	98.38 ± 0.004	98.66 ± 0.004	**99.57 ± 0.0026**
AA			59.41 ± 0.005	77.36 ± 0.019	77.10 ± 0.0228	91.11 ± 0.080	96.59 ± 0.003	**99.62 ± 0.0029**
κ			0.6862 ± 0.007	0.7941 ± 0.007	0.8143 ± 0.0086	0.9815 ± 0.005	0.9848 ± 0.005	**0.9951 ± 0.0030**

TABLE IV

OA, AA, AND KAPPA (κ) VALUES ON UP DATASET USING 10% OF SAMPLES FOR TRAINING

			Classic approaches			Deep-Learning approaches		
Class	Train	Test	RF [64]	SVM [2]	Ensemble-Watershed [49]	SSRN [26]	A2S2K [28]	Triplet-Watershed
1	663	5968	91.11 ± 0.007	94.30 ± 0.008	94.34 ± 0.0032	99.85 ± 0.001	99.91 ± 0.000	**100.0 ± 0.000**
2	1864	16785	98.11 ± 0.003	97.65 ± 0.002	95.24 ± 0.0051	99.98 ± 0.000	99.99 ± 0.000	**100.0 ± 0.000**
3	209	1890	67.71 ± 0.014	81.26 ± 0.018	69.39 ± 0.0151	99.68 ± 0.003	99.88 ± 0.001	**99.8 ± 0.004**
4	306	2758	88.20 ± 0.006	94.63 ± 0.004	78.69 ± 0.0058	99.92 ± 0.001	99.95 ± 0.001	**99.96 ± 0.001**
5	134	1211	98.93 ± 0.002	99.20 ± 0.002	87.46 ± 0.0110	99.94 ± 0.000	**100.0 ± 0.000**	**100.0 ± 0.000**
6	502	4527	72.14 ± 0,022	90.58 ± 0,008	61.37 ± 0.0111	99.95 ± 0.001	99.91 ± 0,001	**99.99 ± 0.001**
7	133	1197	75.69 ± 0.017	85.71 ± 0.011	75.49 ± 0.0295	**100.0 ± 0.000**	**100.0 ± 0.000**	**100.0 ± 0.000**
8	368	3314	89.64 ± 0.013	88.20 ± 0.003	74.65 ± 0.0044	98.28 ± 0.015	98.88 ± 0.006	**99.97 ± 0.001**
9	94	853	99.77 ± 0.002	99.84 ± 0.001	99.77 ± 0.0015	99.39 ± 0.003	99.78 ± 0.003	**100.0 ± 0.000**
OA	4273	38503	90.41 ± 0.001	94.19 ± 0.002	86.13 ± 0.0023	99.77 ± 0.001	99.85 ± 0.001	**99.98 ± 0.001**
AA			86.81 ± 0.002	92.38 ± 0.003	81.82 ± 0.0039	99.66 ± 0.001	99.81 ± 0.001	**99.97 ± 0.001**
κ			0.8710 ± 0.002	0.9229 ± 0.002	0.8136 ± 0.0030	0.9969 ± 0.001	0.9981 ± 0.001	**0.9998 ± 0.001**

TABLE V

OA, AA, AND KAPPA(κ) VALUES ON KSC DATASET USING 10% OF SAMPLES FOR TRAINING

			Classic approaches			Deep-Learning approaches		
Class	Train	Test	RF [64]	SVM [2]	Ensemble-Watershed [49]	SSRN [26]	A2S2K [28]	Triplet-Watershed
1	76	685	94.79 ± 0.012	95.43 ± 0.023	96.23 ± 0.0085	99.95 ± 0.001	99.95 ± 0.001	**100.0 ± 0.0000**
2	24	219	81.58 ± 0 047	83.71 ± 0.012	89.59 ± 0.0247	**100.0 ± 0.000**	98.68 ± 0.019	**100.0 ± 0.0000**
3	25	231	86.09 ± 0 020	78.41 ± 0.218	83.98 ± 0.0341	99.66 ± 0.005	98.72 ± 0.012	**100.0 ± 0.0000**
4	25	227	71.22 ± 0.061	27.17 ± 0.173	69.60 ± 0.0406	91.22 ± 0.047	94.27 ± 0.042	**96.56 ± 0.0423**
5	16	145	47.59 ± 0.060	22.99 ± 0.170	65.52 ± 0.0474	**100.0 ± 0.000**	94.46 ± 0.050	99.86 ± 0.0028
6	22	207	48.22 ± 0.014	36.89 ± 0.078	53.33 ± 0.0526	98.45 ± 0.022	99.82 ± 0.003	**99.52 ± 0.0000**
7	10	95	79.43 ± 0 096	87.94 ± 0.027	85.05 ± 0.0234	95.42 ± 0.050	99.61 ± 0.005	**100.0 ± 0.0000**
8	43	388	78.61 ± 0.054	70.19 ± 0.073	91.24 ± 0.0297	99.80 ± 0.003	**100.0 ± 0.000**	**99.90 ± 0.0000**
9	52	468	89.46 ± 0.011	85.33 ± 0.021	93.08 ± 0.0193	**100.0 ± 0.000**	**100.0 ± 0.000**	**100.0 ± 0.0000**
10	40	364	88.43 ± 0.034	78.88 ± 0.069	92.64 ± 0.0150	**100.0 ± 0.000**	**100.0 ± 0.000**	**100.0 ± 0.0000**
11	41	378	95.58 ± 0.014	93.81 ± 0.008	94.44 ± 0.0261	**100.0 ± 0.000**	**100.0 ± 0.000**	**100.0 ± 0.0000**
12	50	453	82.63 ± 0.032	86.98 ± 0.009	86.98 ± 0.0119	**100.0 ± 0.000**	**100.0 ± 0.000**	99.21 ± 0.0159
13	92	835	99.60 ± 0.002	**100.0 ± 0.000**	99.69 ± 0.0022	**100.0 ± 0.000**	**100.0 ± 0.000**	**100.0 ± 0.0000**
OA	516	4695	86.17 ± 0.004	81.27 ± 0.008	89.54 ± 0.0038	99.29 ± 0.004	99.34 ± 0.0008	**99.72 ± 0.0023**
AA			80.25 ± 0.004	72.90 ± 0.021	84.72 ± 0.0038	98.80 ± 0.008	98.88 ± 0.0018	**99.62 ± 0.0032**
κ			0.8459 ± 0.004	0.7909 ± 0.009	0.8834 ± 0.0042	0.9921 ± 0.004	0.9927 ± 0.001	**0.9969 ± 0.0026**

Remark on Evaluation: Different kinds of evaluations are possible—random train/test split or Patch-based evaluation as proposed in [67]. Here, we use the former since: 1) patch-based evaluation does not recommend using connectivity patterns, while watershed classifier is designed to exploit such patterns and 2) irrespective of the evaluation procedure, we remain consistent with baseline methods (A2S2K, SSRN). Hence, the observations in this article still remain valid.

A. Supervised Classification

First, we provide the results of Triplet-Watershed for supervised classification. We compare our approach with standard baselines (SVM [2] and Random Forest [64]), and also with the two recent state-of-art methods SSRN [26] and A2S2K [28]. Tables III–V show the results (OA, AA, κ) obtained. The train test splits per class are described in

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 04:30:08 UTC from IEEE Xplore. Restrictions apply.

TABLE VI

OA, AA, AND KAPPA (κ) VALUES ON IP DATASET USING SEMI-SUPERVISED APPROACHES

Class	Train	Test	S2GCN [33]	SSRN [26]	DC-GCN [34]	Triplet-Watershed
1	30	16	**100.0 ± 0.0000**	93.24 ± 0.0263	**100.00 ± 0.0000**	**100.00 ± 0.0000**
2	30	1398	84.43 ± 0.0250	76.63 ± 0.0596	91.28 ± 0.0360	**91.69 ± 0.0194**
3	30	800	82.87 ± 0.0553	68.78 ± 0.0753	92.88 ± 0.0396	**95.25 ± 0.0610**
4	30	207	93.08 ± 0.0195	87.64 ± 0.0249	98.11 ± 0.0151	**100.00 ± 0.0000**
5	30	453	97.13 ± 0.0134	86.72 ± 0.0154	95.54 ± 0.0339	**98.63 ± 0.0171**
6	30	700	97.29 ± 0.0127	92.05 ± 0.0182	98.67 ± 0.0104	**100.00 ± 0.0000**
7	15	13	92.31 ± 0.0000	95.66 ± 0.0051	**100.00 ± 0.0000**	**100.00 ± 0.0000**
8	30	448	99.03 ± 0.0093	95.90 ± 0.0297	**100.00 ± 0.0000**	**100.00 ± 0.0000**
9	15	5	**100.00 ± 0.0000**	**100.00 ± 0.0000**	**100.00 ± 0.0000**	**100.00 ± 0.0000**
10	30	942	93.77 ± 0.0373	82.42 ± 0.0324	91.91 ± 0.0378	**98.22 ± 0.0232**
11	30	2425	84.98 ± 0.0282	82.23 ± 0.0288	91.79 ± 0.0379	**94.43 ± 0.0229**
12	30	563	80.05 ± 0.0517	69.09 ± 0.0436	90.17 ± 0.0554	**99.08 ± 0.0185**
13	30	175	99.43 ± 0.0000	95.78 ± 0.0075	99.65 ± 0.0027	**100.00 ± 0.0000**
14	30	1235	96.73 ± 0.0092	86.52 ± 0.0243	99.73 ± 0.0066	**99.87 ± 0.0026**
15	30	356	86.80 ± 0.0342	73.12 ± 0.0528	99.94 ± 0.0016	**100.00 ± 0.0000**
16	30	63	**100.00 ± 0.0000**	86.21 ± 0.0130	**100.00 ± 0.0000**	99.37 ± 0.0078
OA			89.4 ± 0.0108	88.34 ± 0.0173	94.65 ± 0.1210	**96.74 ± 0.0194**
AA			92.9 ± 0.0104	85.75 ± 0.0069	96.85 ± 0.0040	**98.53 ± 0.0098**
κ			0.880 ± 0.012	0.866 ± 0.019	0.944 ± 0.014	**0.9627 ± 0.0221**

TABLE VII

OA, AA, AND KAPPA (κ) VALUES ON UP DATASET USING SEMI-SUPERVISED APPROACHES

Class	Train	Test	S2GCN [33]	SSRN [26]	DC-GCN [34]	Triplet-Watershed
1	30	6601	92.78 ± 0.0379	98.80 ± 0.0110	92.85 ± 0.0351	**99.56 ± 0.0088**
2	30	18619	87.06 ± 0.0447	98.45 ± 0.0054	97.53 ± 0.0140	**100.00 ± 0.0000**
3	30	2069	87.97 ± 0.0477	77.05 ± 0.1024	97.94 ± 0.0118	**99.85 ± 0.0084**
4	30	3034	90.85 ± 0.0094	83.02 ± 0.0907	94.57 ± 0.0109	**99.99 ± 0.0003**
5	30	1315	**100.00 ± 0.0000**	99.96 ± 0.0009	99.49 ± 0.0068	**100.00 ± 0.0000**
6	30	4999	88.69 ± 0.0264	87.03 ± 0.0626	98.57 ± 0.0278	**99.99 ± 0.0001**
7	30	1300	98.88 ± 0.0108	83.92 ± 0.0897	**100.00 ± 0.0000**	**100.00 ± 0.0000**
8	30	3652	89.97 ± 0.0328	88.41 ± 0.0463	**96.00 ± 0.0277**	92.15 ± 0.1560
9	30	917	98.89 ± 0.0053	99.97 ± 0.0004	97.51 ± 0.0140	**100.00 ± 0.0000**
OA			89.74 ± 0.0170	92.81 ± 0.0190	96.87 ± 0.0111	**99.20 ± 0.0129**
AA			92.80 ± 0.0047	90.73 ± 0.0226	97.16 ± 0.0076	**98.95 ± 0.0165**
κ			0.8665 ± 0.020	0.9059 ± 0.024	0.9677 ± 0.012	**0.9894 ± 0.0170**

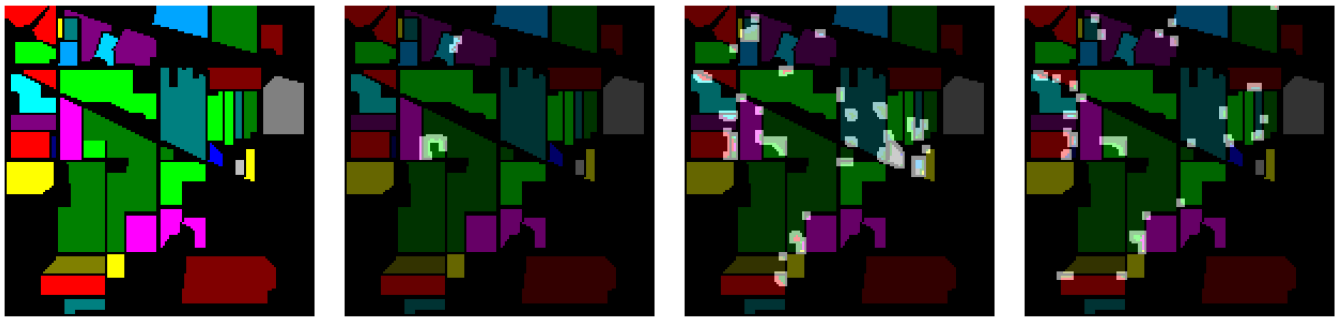
TABLE VIII

OA, AA, AND KAPPA (κ) VALUES ON UH DATASET USING 10% OF SAMPLES FOR TRAINING

			Classic approaches			Deep-Learning approaches		
Class	Train	Test	RF [64]	SVM [2]	Ensemble-Watershed [49]	SSRN [26]	A2S2K [28]	Triplet-Watershed
1	125	1126	82.52 ± 0.0000	82.33 ± 0.0000	93.68 ± 0.0279	99.66 ± 0.0012	**99.79 ± 0.0021**	98.99 ± 0.0080
2	125	1129	83.30 ± 0.0011	83.36 ± 0.0000	81.97 ± 0.0191	99.96 ± 0.0004	**100.0 ± 0.0000**	**100.0 ± 0.0000**
3	69	628	97.62 ± 0.0000	99.80 ± 0.0000	99.90 ± 0.0013	**100.0 ± 0.0000**	**100.0 ± 0.0000**	**100.0 ± 00000**
4	124	1120	91.41 ± 0.0027	98.95 ± 0.0000	74.27 ± 0.0240	99.66 ± 0.0046	99.17 ± 0.0095	**100.0 ± 0.0000**
5	124	1118	96.49 ± 0.0020	98.76 ± 0.0000	82.15 ± 0.0214	**100.0 ± 0.0000**	**100.0 ± 0.0000**	**100.0 ± 00000**
6	32	293	99.30 ± 0.0000	97.90 ± 0.0000	92.22 ± 0.0613	**100.0 ± 0.0000**	**100.0 ± 0.0000**	99.43 ± 0.0080
7	126	1142	75.09 ± 0.0020	77.42 ± 0.0000	69.63 ± 0.0272	99.10 ± 0.0119	98.98 ± 0.0088	**99.65 ± 0.0050**
8	124	1120	33.04 ± 0.0020	60.30 ± 0.0000	78.25 ± 0.0242	99.38 ± 0.0016	**99.72 ± 0.0038**	96.25 ± 0.0338
9	125	1127	69.31 ± 0.0042	76.77 ± 0.0000	52.56 ± 0.0159	**99.30 ± 0.0052**	98.47 ± 0.0101	97.96 ± 0.0145
10	122	1105	44.11 ± 0.0034	61.29 ± 0.0000	63.66 ± 0.0207	94.85 ± 0.0152	94.90 ± 0.0178	**100.0 ± 0.0000**
11	123	1112	70.20 ± 0.0020	80.55 ± 0.0000	56.83 ± 0.0379	99.23 ± 0.0075	**99.42 ± 0.0040**	99.07 ± 0.0131
12	123	1110	54.81 ± 0.0036	79.92 ± 0.0000	54.77 ± 0.0319	98.76 ± 0.0028	99.46 ± 0.0033	**99.64 ± 0.0000**
13	46	423	60.23 ± 0.0129	70.87 ± 0.0000	06.52 ± 0.0130	**99.90 ± 0.0013**	99.01 ± 0.0101	98.74 ± 0.0089
14	42	386	99.32 ± 0.0019	100.0 ± 0.0000	94.15 ± 0.0089	98.63 ± 0.0193	**100.0 ± 0.0000**	**100.0 ± 0.0000**
15	66	594	97.25 ± 0.0017	96.40 ± 0.0000	98.55 ± 0.0051	**100.0 ± 0.0000**	**100.0 ± 0.0000**	**100.0 ± 00000**
OA			73.02 ± 0.0004	81.86 ± 0.0000	72.50 ± 0.0030	99.10 ± 0.0013	99.12 ± 0.0030	**99.25 ± 0.0039**
AA			76.93 ± 0.0004	84.31 ± 0.0000	73.27 ± 0.0046	99.23 ± 0.0016	99.26 ± 0.0020	**99.32 ± 0.0031**
Kappa			71.01 ± 0.0003	80.42 ± 0.0000	70.22 ± 0.0033	99.03 ± 0.0015	99.05 ± 0.0033	**99.19 ± 0.0042**

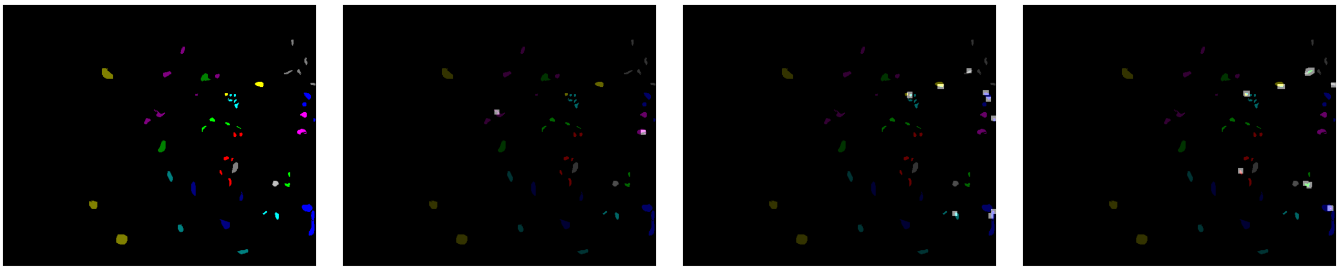
these tables. Note that Triplet-Watershed outperforms existing state-of-art A2S2KResNet [28] and other approaches in several aspects. This can be attributed to the fact that Triplet Watershed exploits the connectivity patterns (edges within the pixels) in the dataset to propagate labels. Other approaches treat each pixel as a separate entity which would not exploit this observation. Other approaches treat each pixel as a separate entity which would not exploit this

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 04:30:08 UTC from IEEE Xplore. Restrictions apply.



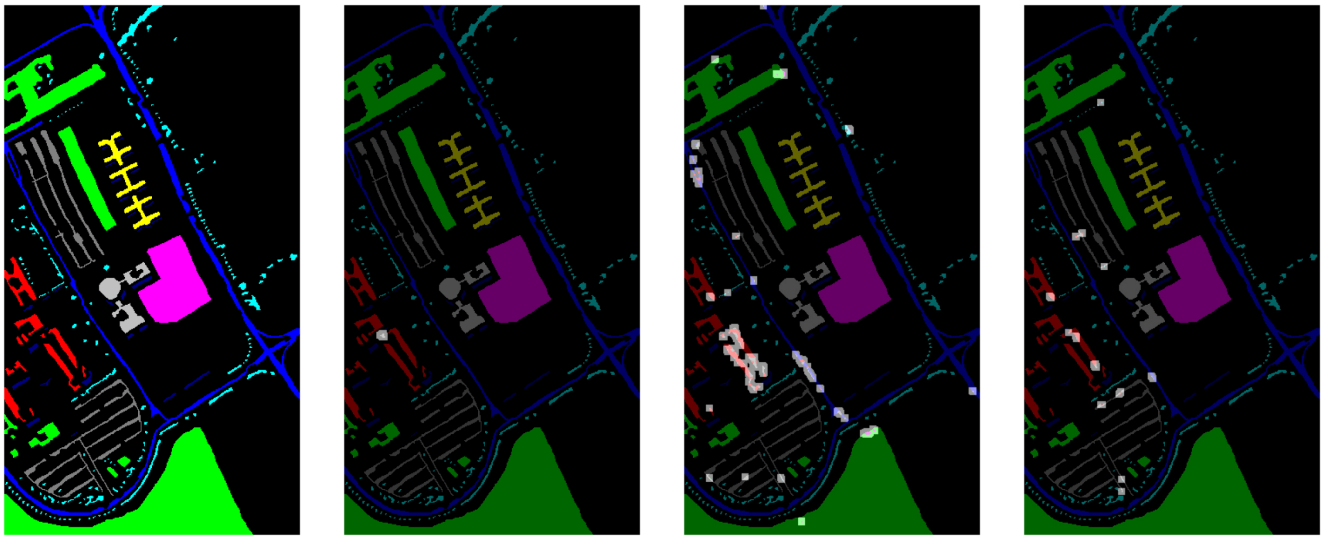
(a) (b) (c) (d)

Fig. 6. Classification maps for IP dataset. The main differences with respect to groundtruth have been highlighted. As one can observe, the number of errors of Triplet-Watershed is small compared to SSRN and A2S2K. (a) GT. (b) Triplet-watershed. (c) SSRN. (d) A2S2KResNet.



(a) (b) (c) (d)

Fig. 7. Classification maps for KSC dataset. The main differences with respect to groundtruth have been highlighted. As one can observe, the number of errors of Triplet-watershed is small compared to SSRN and A2S2K. (a) GT. (b) Triplet-watershed. (c) SSRN. (d) A2S2KResNet.



(a) (b) (c) (d)

Fig. 8. Classification maps for UP dataset. The main differences with respect to groundtruth have been highlighted. As one can observe, the number of errors of Triplet-Watershed is small compared to SSRN and A2S2K. (a) GT. (b) Triplet-watershed. (c) SSRN. (d) A2S2KResNet.

observation. Simple Ensemble-Watershed results are shown in the tables as well. Classification maps for Triplet-Watershed along with competing approaches are shown in Figs. 6–9. High resolution stand-alone images can also be found in https://github.com/ac20/TripletWatershed_Code/tree/main/img/classification_maps.

B. Semi-Supervised Classification

We compare the Triplet-Watershed with three recent state-of-art semi-supervised approaches—S2GCN [33], SSRN [26], and DC-GCN [34]. We consider 30 samples for training if the class size is greater than 30 and 15 if the class size is less than 30. Tables VI and VII show the results obtained. Observe that, once again, Triplet-Watershed obtains the state of the art in several aspects.

C. Evaluation of Representation

Recall that accuracies in Tables III–VIII for Triplet-Watershed use ensemble watershed classifier. However, ensemble watershed exploits the connectivity

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 04:30:08 UTC from IEEE Xplore. Restrictions apply.

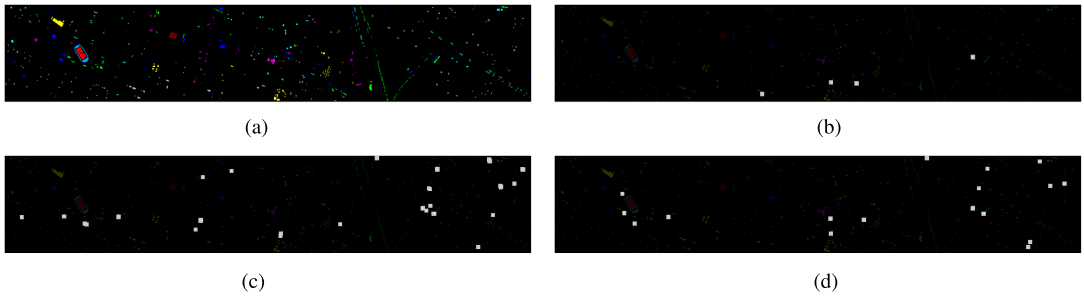


Fig. 9. Classification maps for UH dataset. The main differences with respect to groundtruth have been highlighted. As one can observe, the number of errors of Triplet-Watershed is small compared to SSRN and A2S2K. (a) GT. (b) Triplet-watershed. (c) SSRN. (d) A2S2KResNet.

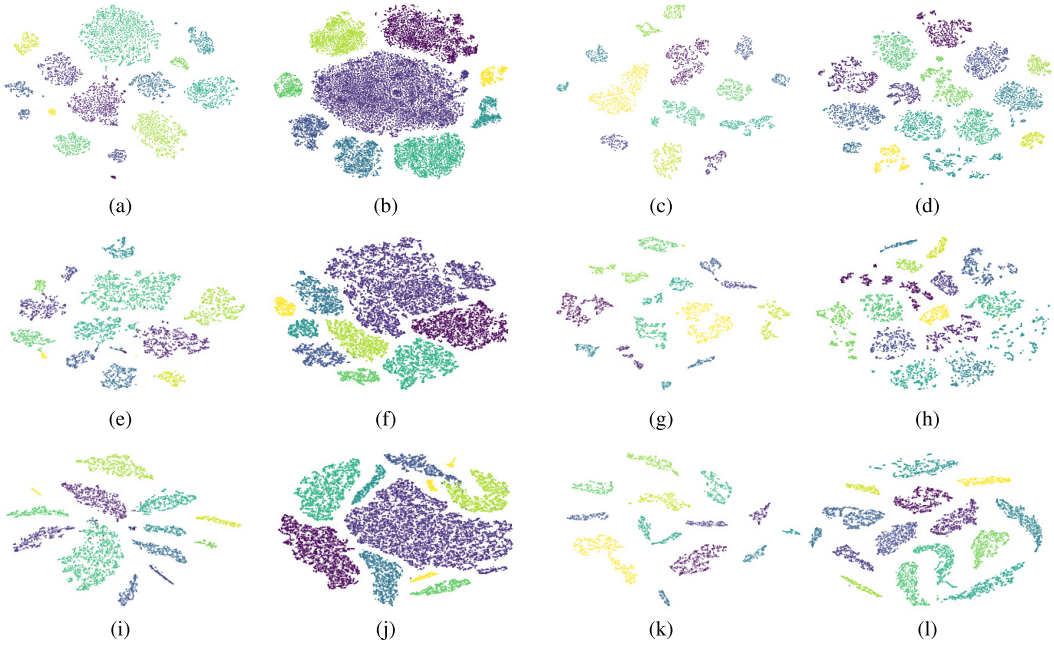


Fig. 10. T-SNE Scatterplot of the various representations obtained. All approaches provide well-separated clusters, relatively compact. Table IX however shows that triplet-watershed achieves a better precision (MAP score). IP denotes Indianpines dataset, UP denotes University of Pavia dataset, KSC denotes Kennedy Space Centre dataset and UH denotes University of Houston dataset. (a) Triplet-watershed-IP. (b) Triplet-watershed-UP. (c) Triplet-watershed-KSC. (d) Triplet-watershed-UH. (e) A2S2K-IP. (f) A2S2K-UP. (g) A2S2K-KSC. (h) A2S2K-UH. (i) SSRN-IP. (j) SSRN-UP. (k) SSRN-KSC. (l) SSRN-UH.

patterns in the data. We now try to understand how well watershed representations compare with representations obtained by other approaches. Qualitatively, we use the TSNE [68] plots as in Fig. 10. Note that there does not exist any major differences except that within a class, A2S2K and SSRN have "clumps" point while Triplet-Watershed has a uniform density. Quantitatively, we use the mean average precision (MAP) over all points. The computation procedure is as follows.

1) Given a data point x_k, we order all other data points $\{y_i\}_i$ using an inverse function of distance, exp(−distance).
2) Labels are assigned based on whether the points $\{y_i\}_i$ belong to the same class as x_k or not with class label 1 and 0, respectively.
3) Average precision (AP) computes the area under the precision-recall curve.
4) The AP scores are averages over all points $\{x_k\}_k$ to obtain the MAP score.

This procedure is as suggested in [69] to evaluate representations. The results are shown in Table IX. Observe that the watershed outperforms the current state-of-the-art techniques.

TABLE IX

MAP SCORES FOR THE REPRESENTATIONS. OBSERVE THAT TRIPLET-WATERSHED OBTAINS BETTER REPRESENTATIONS THAN COMPETING APPROACHES ON ALL DATASETS

	Triplet-Watershed	A2S2K [28]	SSRN [26]
IN	**0.9819**	0.9713	0.9135
UP	**0.9970**	0.9821	0.9703
KSC	**0.9822**	0.9837	0.9846
UH	**0.9821**	0.9799	0.9692

D. Ablation Study

We now study the importance of various aspects of Triplet-Watershed for the accuracies.

1) Accuracy Versus % Training Data: Fig. 11 shows the plots of OA versus % training data. For IP and UP datasets, it can be seen that Triplet-Watershed outperforms other approaches even at small sizes of training data. This can be attributed to the fact that the watershed classifier propagates

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 04:30:08 UTC from IEEE Xplore. Restrictions apply.

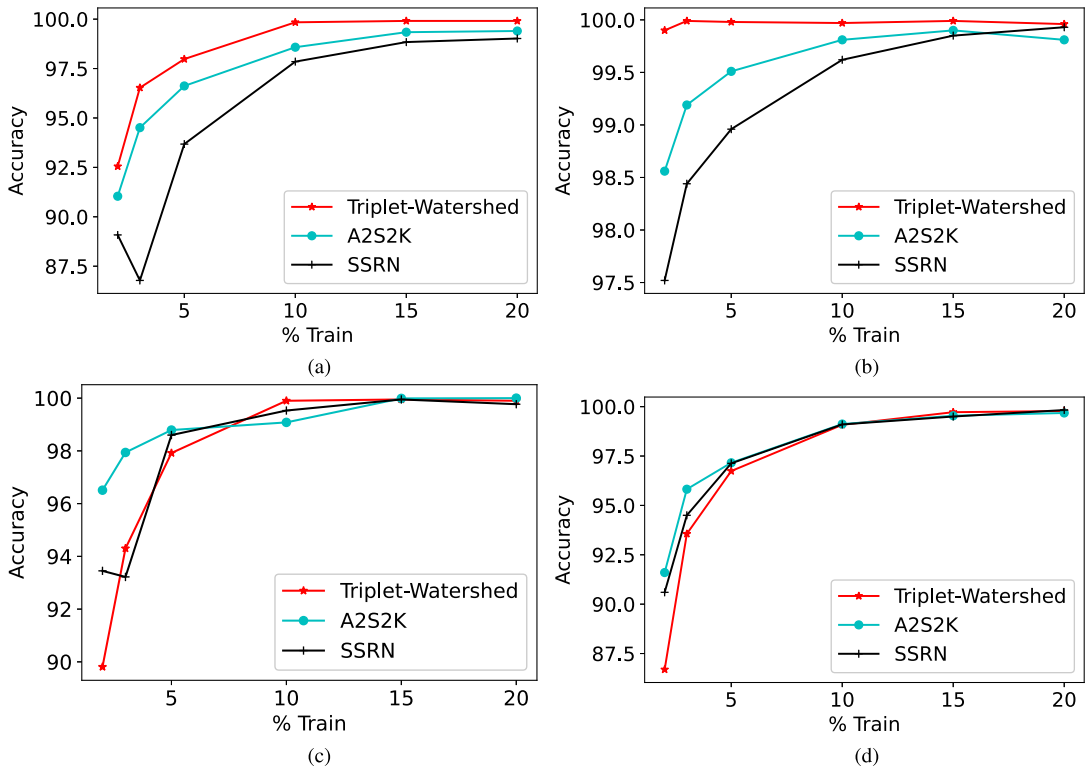


Fig. 11. OA versus % training data. We observe that triplet-watershed outperforms other approaches even at small sizes of training data for IP and UP Dataset. IP denotes Indianpines dataset, UP denotes University of Pavia dataset, KSC denotes Kennedy Space Centre dataset and UH denotes University of Houston dataset. (a) IP. (b) UP. (c) KSC. (d) UH.

TABLE X

COMPARISON OF TRIPLET-WATERSHED WITH TRIPLET-RANDOM-FOREST AND TRIPLET-K-NEAREST-NEIGHBORS. REPLACE WATERSHED CLASSIFIER WITH RANDOM FOREST AND KNN CLASSIFIER TO UNDERSTAND THE IMPORTANCE OF WATERSHED CLASSIFIER

	Triplet-Watershed	Triplet-RF	Triplet-KNN
IN	**99.57 ± 0.0026**	91.46 ± 0.011	90.86 ± 0.013
UP	**99.98 ± 0.001**	98.06 ± 0.007	99.62 ± 0.000
KSC	**99.72 ± 0.0023**	87.80 ± 0.039	82.38 ± 0.031
UH	**99.25 ± 0.004**	89.02 ± 0.018	96.15 ± 0.0086

TABLE XI

TRIPLET-WATERSHED: ACCURACY VERSUS EMBED DIMENSION. NOTE THAT DIFFERENCES ACROSS VARIOUS EMBEDDING DIMENSIONS ARE NOT SIGNIFICANT

Dimension	KSC	IN	UP	UH
16	99.53 ± 0.0031	99.45 ± 0.0025	99.95 ± 0.0002	98.74 ± 0.0034
32	99.70 ± 0.0029	99.72 ± 0.0010	99.97 ± 0.0003	98.73 ± 0.0018
64	99.54 ± 0.0017	99.67 ± 0.0011	99.98 ± 0.0001	99.25 ± 0.0039
128	99.72 ± 0.0004	99.84 ± 0.0009	99.97 ± 0.0001	98.87 ± 0.0025

the information to unlabelled nodes, which is in turn used for training (see Fig. 4). For optimal performance, the watershed classifier requires at least one labeled node per component. In cases of very small training data and many components, Triplet-Watershed does not perform well. This is the case for the KSC dataset at 2% and 3% training data, as shown in Fig. 11. Detailed analysis of the underperformance of Triplet-Watershed at low train sizes for KSC and UH dataset can be found in Appendix B.

2) Replacing Watershed With Other Classifiers: To illustrate the importance of the watershed classifier in the training pipeline (see Fig. 4), we replace it with Random Forest (RF) classifier and K-Nearest Neighbors (KNN) classifier with $k = 5$, referring to these as Triplet-Random Forest and Triplet-K-Nearest-Neighbors. The results are shown in Table X. First, observe the dramatic improvement of accuracies with respect to vanilla classifiers (see Tables III–V). Also, observe that Triplet-Watershed outperforms the other techniques. This, once again, is attributed to the fact that watershed exploits the observation that classes in the groundtruth consist of connected components.

Remark: Both Random Forest (RF) and K-Nearest Neighbors (KNN) are considered for this experiment since the labels generated by these are not differentiable with respect to the input representations. This property is shared with the watershed classifier. However, multilayered perceptron (MLP) and SVM obtain labels using specific costs and are indeed differentiable with respect to their input representations. Hence, the latter approaches are not considered for comparison.

TABLE XII

TRIPLET-WATERSHED: ACCURACY VERSUS PATCH SIZE

		7	9	11	13
IN	OA	99.72 ± 0.0021	99.56 ± 0.0021	99.63 ± 0.0017	99.63 ± 0.0022
	AA	99.72 ± 0.0024	98.57 ± 0.0180	99.82 ± 0.0012	99.75 ± 0.0009
	κ	0.9968 ± 0.0024	0.9949 ± 0.0023	0.9958 ± 0.0020	0.9957 ± 0.0025
UP	OA	99.96 ± 0.0008	99.98 ± 0.0002	99.99 ± 0.0002	99.98 ± 0.0002
	AA	99.93 ± 0.0012	99.96 ± 0.0005	99.98 ± 0.0004	99.96 ± 0.0005
	κ	0.9994 ± 0.0010	0.9997 ± 0.0003	0.9999 ± 0.0002	0.9997 ± 0.0003
KSC	OA	99.77 ± 0.0023	99.96 ± 0.0010	99.96 ± 0.0010	99.96 ± 0.0010
	AA	99.55 ± 0.0050	99.95 ± 0.0010	99.95 ± 0.0010	99.95 ± 0.0010
	κ	0.9975 ± 0.0025	0.9995 ± 0.0010	0.9995 ± 0.0010	0.9995 ± 0.0010
UH	OA	98.22 ± 0.0024	98.78 ± 0.0014	99.25 ± 0.0011	99.23 ± 0.0031
	AA	98.28 ± 0.0038	98.89 ± 0.0015	99.32 ± 0.0013	99.26 ± 0.0031
	κ	0.9807 ± 0.0026	0.9868 ± 0.0015	0.9919 ± 0.0012	0.9915 ± 0.0034

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 04:30:08 UTC from IEEE Xplore. Restrictions apply.

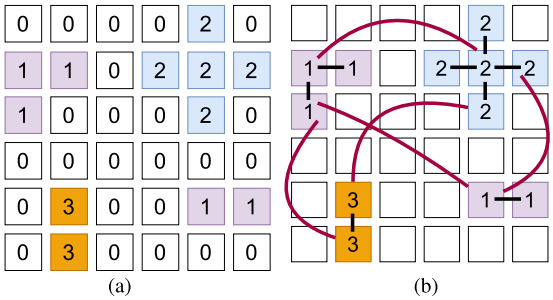


Fig. 12. Constructing the graph on HSI data. (a) Simple toy HSI data with groundtruth classes. Note that class 0 implies that groundtruth is not available. (b) Graph constructed by considering only points with {labels $\neq 0$} as vertices. Four adjacent edges (black) along with other edges (red) spanning across components are considered. These "other" edges are constructed using techniques such as EMST or K-Neighbors graph. (a) Groundtruth. (b) Graph.

3) Accuracy Versus Embed Dimension: Table XI shows the effect of embedding dimension on accuracy. Observe that there does not exist any significant trend with respect to the embedding dimension. We use 64 as the default embedding dimension.

4) Accuracy Versus Patch Size: Recall that one of the hyperparameters of the approach is patch size—the size of the window around the pixel. Table XII shows the results obtained by varying the patch sizes across different datasets. Observe that a larger window size implies more information for inference and hence scope for better inference. Thus, as a rule of thumb, a larger window size obtains better results. But, it also implies a higher computational requirement. However, in several cases increasing the window size beyond a threshold would not lead to significant improvements. For example, in Table XII IN and UP datasets do not show much improvement with larger window sizes. UH dataset improves with larger window size, but no significant improvement is obtained by increasing the window size from 11 to 13.

V. Conclusion

In this article, we proposed a novel approach to train for the watershed classifier. We refer to this as Triplet-Watershed. We show that the watershed classifier exploits the connectivity patterns in the datasets. This leads to huge performance gains compared to other approaches which use simple softmax classifier. We prove this empirically by comparing Triplet-Watershed with existing state-of-art deep learning approaches such as A2S2K [28], SSRN [26] and also classic approaches - RF [64] and SVM [2]. We also compare the current technique with semi-supervised approaches such as S2GCN [33], and dc-GCN [34]. In each case, we achieve better accuracy while using a quarter of the parameters of the previous state-of-the-art approaches.

Appendix A
Constructing the Graph on HSI

Here, we illustrate the process of constructing the graph on HSI dataset. Fig. 12(a) considers a simple hypothetical image with the groundtruth classes as shown. Fig. 12(b) shows the graph obtained using the following steps.

1) Firstly, only points with groundtruth available, i.e., {labels $\neq 0$} are considered. This can be trivially extended to other points depending on requirement. These points constitute the vertex set.
2) The edge set is obtained by taking the union of: 1) four adjacent edges denoted by color black and 2) "other" edges which span across components. These "other" edges are constructed using EMST for IP, UP, and KSC datasets. For the UH dataset these edges are constructed using K-Neighbors graph with $k = 50$.

The two main principles for selecting the graph are: 1) we require each label-induced subgraph[4] such that the number of connected components are as few as possible and 2) we also require the number of edges to be as few as possible. Both these act against each other and the right combination is obtained through trial and error.

Appendix B
Triplet-Watershed at Small Train Sizes

Note that from Fig. 11, at low train sizes (2% and 3%, Triplet-Watershed performs better than A2S2KResNet and SSRN on IP, UP datasets. While, Triplet-Watershed is slightly inferior to A2S2KResNet and SSRN on KSC, UH datasets. In this section, we analyze and explain this in detail.

There are two main reasons for the different behaviors of Triplet-Watershed at high (10%) and low (2%, 3%) train sizes: 1) at low train sizes, not all components within the data are covered and 2) there are not enough points near the boundary to allow for better separation. To understand this better, we perform a post-hoc analysis on UH and IP datasets.

For each label (both groundtruth and prediction), we consider the subgraph induced by the vertices[5] of the given label. In this subgraph, we count the size of each connected component. Table XIII shows these values for UH/IP datasets, for groundtruth labels, and labels predicted for 10% and 2%. Both the above phenomenon can be observed in Table XIII.

1) Observe that for several classes in the UH dataset, there exist small components for UH (example: class 1 with 178 points) which are not represented when only 2% of the data is considered for training. While, this happens for the IP dataset (class 5, 147 points), it is relatively low in magnitude. This partly explains why we achieve better results at 10% train size. And also why IP performs better at 2% train size comparatively.
2) The other main reason is—boundaries are not sufficiently represented at 2% train size. As an example of this, consider class 13 for the UH dataset which has a single component 469 points. At 2% train size, this component splits into small components. However, at 10% train size, the component is preserved. This is due to insufficient boundary information at 2% train size. Moreover, as can be intuitively expected, this happens when there is a relatively high standard deviation within the class.

[4]Given a graph $G = (V, E, W)$, the subgraph induces by a subset of vertices $V' \subset V$ is given by $G' = (V', E', W)$. Here, $E' = \{(e_x, e_y) \in E$ such that $e_x, e_y \in V'\}$.

[5]See footnote 4.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 04:30:08 UTC from IEEE Xplore. Restrictions apply.

TABLE XIII

SIZES OF COMPONENTS OF LABEL-INDUCED SUBGRAPHS FOR DATASETS UH AND IP. THREE KINDS OF LABELS ARE CONSIDERED-GROUNDTRUTH, LABELS PREDICTED AT 2% AND 10%. ALSO, SHOWN ARE THE RELATIVE (TO MAXIMUM) STANDARD DEVIATIONS OF THE GROUNDTRUTH COMPONENTS

		UH					IP		
Label	Groundtruth	Rel. Stdev.	10%	2%	Label	Groundtruth	Rel. Stdev.	10%	2%
1	[178, 1073]	[0.41, 0.68]	[154, 1073]	[1073]	1	[46]	[0.79]	[46]	[46]
2	[1096, 158]	[0.68, 0.4]	[1096, 158]	[312, 1130]	2	[1428]	[0.76]	[1416, 1, 1]	[1440]
3	[697]	[0.47]	[697]	[697]	3	[830]	[0.73]	[830]	[830]
4	[1174, 70]	[0.61, 0.39]	[1268]	[1268]	4	[237]	[0.83]	[237]	[237]
5	[1242]	[0.72]	[1272]	[1315]	5	[318,147,18]	[0.65, 0.74, 0.72]	[318, 147, 18]	[318, 88]
6	[40, 6, 279]	[0.51, 0.64, 0.65]	[40, 6, 279]	[6, 279]	6	[730]	[0.68]	[730]	[730]
7	[1268]	[0.56]	[1268]	[1225]	7	[28]	[0.68]	[28]	[28]
8	[1011, 170, 34, 20, 9]	[0.93, 0.42, 0.66, 0.56, 0.63]	[998, 170, 34, 20]	[290, 224, 476]	8	[478]	[0.85]	[478]	[478]
9	[1243, 9]	[0.66, 0.64]	[1257, 9]	[1302, 9, 1]	9	[20]	[0.67]	[20]	[20]
10	[901, 326]	[0.65, 0.42]	[905, 326]	[908, 368]	10	[912, 60]	[0.69, 0.84]	[912, 60]	[912, 60]
11	[1235]	[0.56]	[1204]	[1176, 118]	11	[2455]	[0.74]	[2465]	[2520]
12	[1233]	[0.65]	[1237]	[1726]	12	[593]	[0.82]	[633]	[624]
13	[469]	[1.0]	[461]	[3, 13, 1, 1]	13	[205]	[0.68]	[205]	[205]
14	[428]	[0.95]	[428]	[428]	14	[1265]	[0.74]	[1265]	[1265]
15	[660]	[0.67]	[669]	[680]	15	[386]	[0.76]	[386]	[386]
					16	[93]	[1.0]	[53]	[62]

The above observations explain the behavior of Triplet-Watershed at low train sizes.

ACKNOWLEDGMENT

The authors would like to thank the Associate-Editor, the Editor-in-Chief, and the anonymous reviewers for their valuable comments.

REFERENCES

[1] P. Ghamisi *et al.*, "Advances in hyperspectral image and signal processing: A comprehensive overview of the state of the art," *IEEE Geosci. Remote Sens. Mag.*, vol. 5, no. 4, pp. 37–78, Dec. 2017.

[2] F. Melgani and L. Bruzzone, "Classification of hyperspectral remote sensing images with support vector machines," *IEEE Trans. Geosci. Remote. Sens.*, vol. 42, no. 8, pp. 1778–1790, Aug. 2004, doi: 10.1109/TGRS.2004.831865.

[3] P. O. Gislason, J. A. Benediktsson, and J. R. Sveinsson, "Random forests for land cover classification," *Pattern Recognit. Lett.*, vol. 27, no. 4, pp. 294–300, 2006, doi: 10.1016/j.patrec.2005.08.011.

[4] Y. Cai, X. Liu, and Z. Cai, "BS-Nets: An end-to-end framework for band selection of hyperspectral image," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 3, pp. 1969–1984, Mar. 2020, doi: 10.1109/TGRS.2019.2951433.

[5] S. K. Roy, S. Das, T. Song, and B. Chanda, "DARecNet-BS: Unsupervised dual-attention reconstruction network for hyperspectral band selection," *IEEE Geosci. Remote Sens. Lett.*, early access, Aug. 11, 2020, doi: 10.1109/LGRS.2020.3013235.

[6] D. Hong, N. Yokoya, J. Chanussot, J. Xu, and X. X. Zhu, "Joint and progressive subspace analysis (JPSA) with spatial–spectral manifold alignment for semisupervised hyperspectral dimensionality reduction," *IEEE Trans. Cybern.*, vol. 51, no. 7, pp. 3602–3615, Jul. 2021.

[7] D. Hong, N. Yokoya, J. Xu, and X. Zhu, "Joint and progressive learning from high-dimensional data for multi-label classification," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, V. Ferrari, M. Hebert, C. Sminchisescu, and Y. Weiss, Eds. Cham, Switzerland: Springer, 2018, pp. 478–493.

[8] D. Hong *et al.*, "More diverse means better: Multimodal deep learning meets remote-sensing imagery classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 5, pp. 4340–4354, May 2020.

[9] L. Gao, D. Yao, Q. Li, L. Zhuang, B. Zhang, and J. Bioucas-Dias, "A new low-rank representation based hyperspectral image denoising method for mineral mapping," *Remote Sens.*, vol. 9, no. 11, p. 1145, Nov. 2017. [Online]. Available: https://www.mdpi.com/2072-4292/9/11/1145

[10] Y. Li, Q. Li, Y. Liu, and W. Xie, "A spatial–spectral SIFT for hyperspectral image matching and classification," *Pattern Recognit. Lett.*, vol. 127, pp. 18–26, Nov. 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167865518305117

[11] Y. Shao, N. Sang, C. Gao, and L. Ma, "Spatial and class structure regularized sparse representation graph for semi-supervised hyperspectral image classification," *Pattern Recognit.*, vol. 81, pp. 81–94, Sep. 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0031320318301171

[12] G. Licciardi, P. R. Marpu, J. Chanussot, and J. A. Benediktsson, "Linear versus nonlinear PCA for the classification of hyperspectral data based on the extended morphological profiles," *IEEE Geosci. Remote Sens. Lett.*, vol. 9, no. 3, pp. 447–451, May 2012.

[13] A. Villa, J. A. Benediktsson, J. Chanussot, and C. Jutten, "Hyperspectral image classification with independent component discriminant analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 49, no. 12, pp. 4865–4876, Dec. 2011.

[14] Y. Zhong and L. Zhang, "An adaptive artificial immune network for supervised classification of multi-/hyperspectral remote sensing imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 50, no. 3, pp. 894–909, Mar. 2012.

[15] J. Li, J. M. Bioucas-Dias, and A. Plaza, "Semisupervised hyperspectral image segmentation using multinomial logistic regression with active learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 48, no. 11, pp. 4085–4098, Nov. 2010.

[16] P. Ghamisi *et al.*, "New frontiers in spectral–spatial hyperspectral image classification: The latest advances based on mathematical morphology, Markov random fields, segmentation, sparse representation, and deep learning," *IEEE Geosci. Remote Sens. Mag.*, vol. 6, no. 3, pp. 10–43, Sep. 2018.

[17] L. He, J. Li, C. Liu, and S. Li, "Recent advances on spectral–spatial hyperspectral image classification: An overview and new guidelines," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 3, pp. 1579–1597, Mar. 2017.

[18] S. Li, W. Song, L. Fang, Y. Chen, P. Ghamisi, and J. A. Benediktsson, "Deep learning for hyperspectral image classification: An overview," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 6690–6709, Sep. 2019.

[19] G. Camps-Valls, L. Gomez-Chova, J. Munoz-Mari, J. Vila-Frances, and J. Calpe-Maravilla, "Composite kernels for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 3, no. 1, pp. 93–97, Jan. 2006.

[20] M. Fauvel, J. Chanussot, and J. A. Benediktsson, "A spatial–spectral kernel-based approach for the classification of remote-sensing images," *Pattern Recognit.*, vol. 45, no. 1, pp. 381–392, 2012. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0031320311002019

[21] L. Fang, S. Li, W. Duan, J. Ren, and J. A. Benediktsson, "Classification of hyperspectral images by exploiting spectral–spatial information of superpixel via multiple kernels," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 12, pp. 6663–6674, Dec. 2015.

[22] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Oct. 2016.

[23] J. Yang, Y.-Q. Zhao, and J. C.-W. Chan, "Learning and transferring deep joint spectral–spatial features for hyperspectral classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 8, pp. 4729–4742, Aug. 2017.

[24] A. B. Hamida, A. Benoit, P. Lambert, and C. B. Amar, "3-D deep learning approach for remote sensing image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 8, pp. 4420–4434, Aug. 2018.

[25] M. He, B. Li, and H. Chen, "Multi-scale 3D deep convolutional neural network for hyperspectral image classification," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Sep. 2017, pp. 3904–3908.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 04:30:08 UTC from IEEE Xplore. Restrictions apply.

[26] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.

[27] M. Zhu, L. Jiao, F. Liu, S. Yang, and J. Wang, "Residual Spectral–spatial attention network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 1, pp. 449–462, Jan. 2021.

[28] S. K. Roy, S. Manna, T. Song, and L. Bruzzone, "Attention-based adaptive spectral–spatial kernel ResNet for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 9, pp. 7831–7843, Sep. 2020.

[29] D. Hong, L. Gao, J. Yao, B. Zhang, A. Plaza, and J. Chanussot, "Graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 5966–5978, Jul. 2021.

[30] M. E. Paoletti *et al.*, "Capsule networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 4, pp. 2145–2160, Apr. 2019.

[31] D. Hong, N. Yokoya, N. Ge, J. Chanussot, and X. X. Zhu, "Learnable manifold alignment (LeMA): A semi-supervised cross-modality learning framework for land cover and land use classification," *ISPRS J. Photogramm. Remote Sens.*, vol. 147, pp. 193–205, Jan. 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0924271618302843

[32] D. Hong, N. Yokoya, J. Chanussot, and X. X. Zhu, "CoSpace: Common subspace learning from hyperspectral-multispectral correspondences," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 7, pp. 4349–4359, Jul. 2019.

[33] A. Qin, Z. Shang, J. Tian, Y. Wang, T. Zhang, and Y. Y. Tang, "Spectral–spatial graph convolutional networks for semisupervised hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 2, pp. 241–245, Sep. 2019.

[34] H. Zeng, Q. Liu, M. Zhang, X. Han, and Y. Wang, "Semi-supervised hyperspectral image classification with graph clustering convolutional networks," 2020, *arXiv:2012.10932*. [Online]. Available: http://arxiv.org/abs/2012.10932

[35] Y. Duan, H. Huang, and Y. Tang, "Local constraint-based sparse manifold hypergraph learning for dimensionality reduction of hyperspectral image," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 1, pp. 613–628, Jan. 2021.

[36] H. Huang, Z. Li, H. He, Y. Duan, and S. Yang, "Self-adaptive manifold discriminant analysis for feature extraction from hyperspectral imagery," *Pattern Recognit.*, vol. 107, Nov. 2020, Art. no. 107487. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0031320320302909

[37] Z. Li, H. Huang, Y. Duan, and G. Shi, "DLPNet: A deep manifold network for feature extraction of hyperspectral imagery," *Neural Netw.*, vol. 129, pp. 7–18, Sep. 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0893608020301921

[38] H. Huang, C. Pu, Y. Li, and Y. Duan, "Adaptive residual convolutional neural network for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 2520–2531, May 2020.

[39] L. Vincent and P. Soille, "Watersheds in digital spaces: An efficient algorithm based on immersion simulations," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 13, no. 6, pp. 583–598, Jun. 1991, doi: 10.1109/34.87344.

[40] S. Beucher and F. Meyer, *The Morphological Approach to Segmentation: The Watershed Transformation*, vol. 34. Boca Raton, FL, USA: CRC Press, 1993, pp. 433–481.

[41] G. Noyel, J. Angulo, and D. Jeulin, "Morphological segmentation of hyperspectral images," *Image Anal. Stereol.*, vol. 26, pp. 101–109, Oct. 2007.

[42] Y. Tarabalka, J. Chanussot, and J. A. Benediktsson, "Segmentation and classification of hyperspectral images using watershed transformation," *Pattern Recognit.*, vol. 43, no. 7, pp. 2367–2379, 2010.

[43] J. Cousty, G. Bertrand, L. Najman, and M. Couprie, "Watershed cuts: Minimum spanning forests and the drop of water principle," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 31, no. 8, pp. 1362–1374, Aug. 2009, doi: 10.1109/TPAMI.2008.173.

[44] S. C. Turaga, K. L. Briggman, M. Helmstaedter, W. Denk, and H. S. Seung, "Maximin affinity learning of image segmentation," in *Proc. 23rd Annu. Conf. Neural Inf. Process. Syst.*, Vancouver, BC, Canada, Y. Bengio, D. Schuurmans, J. D. Lafferty, C. K. I. Williams, and A. Culotta, Eds. Red Hook, NY, USA: Curran Associates, Dec. 2009, pp. 1865–1873. [Online]. Available: https://proceedings.neurips.cc/paper/2009/hash/68d30a9594728bc39aa24be94b319d21-Abstract.html

[45] K.-K. Maninis, J. Pont-Tuset, L. Van Gool, and P. Arbeláez, "Convolutional oriented boundaries: From image segmentation to high-level tasks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 4, pp. 819–833, Apr. 2018.

[46] S. Wolf, L. Schott, U. Kothe, and F. Hamprecht, "Learned watershed: End-to-end learning of seeded segmentation," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Venice, Italy, Oct. 2017, pp. 2030–2038, doi: 10.1109/ICCV.2017.222.

[47] J. Funke *et al.*, "Large scale image segmentation with structured loss based deep learning for connectome reconstruction," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 7, pp. 1669–1680, Jul. 2019.

[48] S. Wolf *et al.*, "The mutex watershed and its objective: Efficient, parameter-free graph partitioning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 43, no. 10, pp. 3724–3738, Oct. 2021.

[49] A. Challa, S. Danda, B. S. D. Sagar, and L. Najman, "Watersheds for semi-supervised classification," *IEEE Signal Process. Lett.*, vol. 26, no. 5, pp. 720–724, May 2019, doi: 10.1109/LSP.2019.2905155.

[50] Y. Shen *et al.*, "Efficient deep learning of nonlocal features for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 6029–6043, Jul. 2021.

[51] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 770–778, doi: 10.1109/CVPR.2016.90.

[52] A. X. Falcao, J. Stolfi, and R. D. A. Lotufo, "The image foresting transform: Theory, algorithms, and applications," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 26, no. 1, pp. 19–29, Jan. 2004, doi: 10.1109/TPAMI.2004.10012.

[53] W. P. Amorim, A. X. Falcão, and M. H. de Carvalho, "Semisupervised pattern classification using optimum-path forest," in *Proc. 27th Conf. Graph., Patterns Images (SIBGRAPI)*, Rio de Janeiro, Brazil, Aug. 2014, pp. 111–118, doi: 10.1109/SIBGRAPI.2014.45.

[54] E. Hoffer and N. Ailon, "Deep metric learning using triplet network," in *Proc. 3rd Int. Conf. Learn. Represent. (ICLR)*, Y. Bengio and Y. LeCun, Eds., San Diego, CA, USA, May 2015, pp. 84–92. [Online]. Available: http://arxiv.org/abs/1412.6622

[55] M. Schultz and T. Joachims, "Learning a distance metric from relative comparisons," in *Proc. Adv. Neural Inf. Process. Syst.*, Vancouver, BC, Canada, S. Thrun, L. K. Saul, and B. Schölkopf, Eds. Cambridge, MA, USA: MIT Press, Dec. 2003, pp. 41–48. [Online]. Available: https://proceedings.neurips.cc/paper/2003/hash/d3b1fb02964aa64e257f9f26a31f72cf-Abstract.html

[56] C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals, "Understanding deep learning requires rethinking generalization," in *Proc. 5th Int. Conf. Learn. Represent. (ICLR)*, Toulon, France, Apr. 2017, pp. 1–9. [Online]. Available: https://openreview.net/ forum?id=Sy8gdB9xx

[57] P. W. Battaglia *et al.*, "Relational inductive biases, deep learning, and graph networks," 2018, *arXiv:1806.01261*. [Online]. Available: http://arxiv.org/abs/1806.01261

[58] L. Najman, J. Cousty, and B. Perret, "Playing with Kruskal: Algorithms for morphological trees in edge-weighted graphs," in *Proc. 11th Int. Symp., Math. Morphol. Appl. Signal Image Process.*, in Lecture Notes in Computer Science, vol. 7883, C. L. L. Hendriks, G. Borgefors, and R. Strand, Eds., Uppsala, Sweden. Berlin, Germany: Springer, May 2013, pp. 135–146, doi: 10.1007/978-3-642-38294-9_12.

[59] B. Perret, G. Chierchia, J. Cousty, S. J. F. Guimarães, Y. Kenmochi, and L. Najman, "Higra: Hierarchical graph analysis," *SoftwareX*, vol. 10, Jul. 2019, Art. no. 100335, doi: 10.1016/j.softx.2019.100335.

[60] R. O. Green *et al.*, "Imaging spectroscopy and the airborne visible/infrared imaging spectrometer (AVIRIS)," *Remote Sens. Environ.*, vol. 65, no. 3, pp. 227–248, Sep. 1998. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0034425798000649

[61] B. Kunkel, F. Blechinger, R. Lutz, R. Doerffer, H. van D. Piepen, and M. Schroder, "ROSIS (reflective optics system imaging spectrometer)—A candidate instrument for polar platform missions," in *Proc. Optoelectron. Technol. Remote Sens. Space*, C. S. Bowyer and J. S. Seeley, Eds. Bellingham, WA, USA: SPIE, Apr. 1988, doi: 10.1117/12.943611.

[62] K. Pearson, "On lines and planes of closest fit to systems of points in space," *London, Edinburgh, Dublin Philos. Mag. J. Sci.*, vol. 2, no. 11, pp. 559–572, 1901, doi: 10.1080/14786440109462720.

[63] W. B. March, P. Ram, and A. G. Gray, "Fast Euclidean minimum spanning tree: Algorithm, analysis, and applications," in *Proc. 16th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Washington, DC, USA, B. Rao, B. Krishnapuram, A. Tomkins, and Q. Yang, Eds., New York, NY, USA, Jul. 2010, pp. 603–612, doi: 10.1145/1835804.1835882.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 04:30:08 UTC from IEEE Xplore. Restrictions apply.

[64] J. Ham, Y. Chen, M. M. Crawford, and J. Ghosh, "Investigation of the random forest framework for classification of hyperspectral data," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 3, pp. 492–501, Mar. 2005.
[65] L. N. Smith, "Cyclical learning rates for training neural networks," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Santa Rosa, CA, USA, Mar. 2017, pp. 464–472, doi: 10.1109/WACV.2017.58.
[66] A. Paszke *et al.*, "Pytorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, H. Wallach, H. Larochelle, A. Beygelzimer, F. dá lché-Buc, E. Fox, and R. Garnett, Eds. Red Hook, NY, USA: Curran Associates, 2019, pp. 8024–8035. [Online]. Available:http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library. pdf
[67] J. Nalepa, M. Myller, and M. Kawulok, "Validating hyperspectral image segmentation," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 8, pp. 1264–1268, Aug. 2019.
[68] L. Van D. Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 11, pp. 2579–2605, 2008.
[69] K. Musgrave, S. Belongie, and S.-N. Lim, "A metric learning reality check," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, Aug. 2020, pp. 681–699.



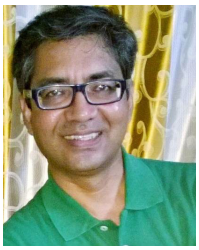
Aditya Challa received the B.Math. degree (Hons.) in mathematics from the Indian Statistical Institute, Bengaluru, India, in 2010, the master's degree in complex systems from the University of Warwick, Coventry, U.K., in 2012, and the Ph.D. degree in computer science from the Systems Science and Informatics Unit, Indian Statistical Institute, Bengaluru, in 2019.

From 2012 to 2014, he worked as a Business Analyst with Tata Consultancy Services, Bengaluru. He has worked as a Raman Post-Doctoral Fellow with the Indian Institute of Science, Bengaluru, from 2019 to 2021. He is currently working as an Assistant Professor with the Department of Computer Science and Information Systems, Birla Institute of Technology and Science (BITS) Pilani, K. K. Birla Goa Campus, Sancoale, Goa, India. His research interests focus on using techniques from mathematical morphology in machine learning.



Sravan Danda (Member, IEEE) received the B.Math. degree (Hons.) in mathematics from the Indian Statistical Institute at Bengaluru, Bengaluru, India, in 2009, the M.Stat. degree in mathematical statistics from the Indian Statistical Institute at Kolkata, Kolkata, India, in 2011, and the Ph.D. degree in computer science from the Systems Science and Informatics Unit, Indian Statistical Institute at Bengaluru, in 2019, under the joint supervision of B. S. Daya Sagar and Laurent Najman.

From 2011 to 2013, he has worked as a Business Analyst with Genpact–Retail Analytics, Bengaluru. He is currently working as an Assistant Professor with the Department of Computer Science and Information Systems, Birla Institute of Technology and Science (BITS) Pilani, K. K. Birla Goa Campus, Sancoale, Goa, India. His research interests include discrete mathematical morphology and discrete optimization.



B. S. Daya Sagar (Senior Member, IEEE) received the M.Sc. and Ph.D. degrees in geoengineering and remote sensing from the Faculty of Engineering, Andhra University, Visakhapatnam, India, in 1991 and 1994, respectively.

He is currently a Full Professor with the Systems Science and Informatics Unit (SSIU), Indian Statistical Institute at Bengaluru, Bengaluru, India, where he is also the first Head of SSIU. He worked with the College of Engineering, Andhra University, and the Centre for Remote Imaging Sensing and Processing (CRISP), National University of Singapore, Singapore, in various positions, from 1992 to 2001. He served as an Associate Professor and a Researcher with the Faculty of Engineering and Technology (FET), Multimedia University, Cyberjaya, Malaysia, from 2001 to 2007. He has made significant contributions to the field of geosciences, with a special emphasis on the development of spatial algorithms meant for geo-pattern retrieval, analysis, reasoning, modeling, and visualization by using concepts of mathematical morphology and fractal geometry. He has published more than 85 articles in journals and has authored and/or guest-edited 11 books and/or special theme issues for journals. He recently authored a book titled *Mathematical Morphology in Geomorphology and GISci* (Boca Raton: CRC Press, 2013, p. 546). He recently co-edited two special issues on "Filtering and Segmentation With Mathematical Morphology" of IEEE JOURNAL OF SELECTED TOPICS IN SIGNAL PROCESSING (v. 6, no. 7, 2012, p. 737–886), and "Applied Earth Observation and Remote Sensing in India" of IEEE JOURNAL OF SELECTED TOPICS IN APPLIED EARTH OBSERVATIONS AND REMOTE SENSING (v. 10, no. 12, 2017, p. 5149–5328). His recent book *Handbook of Mathematical Geosciences* (Springer Publishers, 2018, p. 942) reached 750 000 downloads.

Dr. Daya Sagar was elected as a member of the New York Academy of Sciences in 1995, a fellow of the Royal Geographical Society in 2000, a Senior Member of the IEEE Geoscience and Remote Sensing Society (GRSS) in 2003, and a fellow of the Indian Geophysical Union in 2011. He is also a member of the American Geophysical Union since 2004 and a Life Member of the International Association for Mathematical Geosciences (IAMG). He delivered the "Curzon & Co–Seshachalam Lecture" at Sarada Ranganathan Endowment Lectures (SRELS), Bengaluru, in 2009, and the "Frank Harary Endowment Lecture" at International Conference on Discrete Mathematics-2019 (ICDM-2019), in 2019. He received the Dr. Balakrishna Memorial Award of the Andhra Pradesh Academy of Sciences in 1995, the Krishnan Medal of the Indian Geophysical Union in 2002, the Georges Matheron Award with Lectureship of the IAMG in 2011, and the Award of IAMG Certificate of Appreciation in 2018. He is the Founding Chairman of the Bangalore Section IEEE GRSS Chapter. He has been recently appointed as an IEEE GRSS Distinguished Lecturer (DL) for a two-year period from 2020 to 2022. He is on the Editorial Boards of *Computers and Geosciences*, *Environmental Informatics* (Frontiers), and *Mathematical Geosciences*. He is also the Editor-In-Chief of the *Encyclopedia of Mathematical Geosciences* (Springer).



Laurent Najman (Senior Member, IEEE) received the "Ingénieur" degree from the Ecole des Mines de Paris, Paris, France, in 1991, the Ph.D. degree (Hons.) in applied mathematics from Paris-Dauphine University, Paris, in 1994, and the Habilitation à Diriger les Recherches from the University of Marne-la-Vallée, Champs-sur-Marne, France, in 2006.

After earning his engineering degree, he has worked in the Central Research Laboratories of Thomson-CSF, Corbeville, France, for three years, working on some problems of infrared image segmentation using mathematical morphology. In 1998, he joined OCÉ Print Logic Technologies, Créteil, France, as the Senior Scientist. He has worked there on various problem of image analysis dedicated to scanning and printing. In 2002, he joined the Informatics Department of ESIEE, Paris, where he is currently a Professor and a member of the Institut Gaspard Monge, Université Gustave Eiffel. His research interests include discrete mathematical morphology and discrete optimization.

Dr. Najman joined a startup company named Animation Science, Paris, in 1995, as the Director of research and development. The technology of particle systems for computer graphics and scientific visualization, developed by the company under his technical leadership received several awards, including the "European Information Technology Prize 1997" awarded by the European Commission (Esprit Programme) and by the European Council for Applied Science and Engineering and the "Hottest Products of the Year 1996" awarded by the *Computer Graphics World* journal.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 09,2026 at 04:30:08 UTC from IEEE Xplore. Restrictions apply.